

Estudiante: Edgar Eliam Araya Alvarado

Aplicaciones Web Utilizando Software Libre

Proyeccto-1 Laravel From Scratch 2026

Entregable #1

Fecha 22/06/2026

Misael Matamoros Soto

Routing 101

¿Qué es una ruta en Laravel?

En Laravel, una **ruta** define qué debe ocurrir cuando un usuario visita una URL específica de la aplicación.

La sintaxis de una ruta es fácil de interpretar porque se parece al lenguaje natural:

```
Route::get('/', function () {  
    return view('welcome');  
});
```

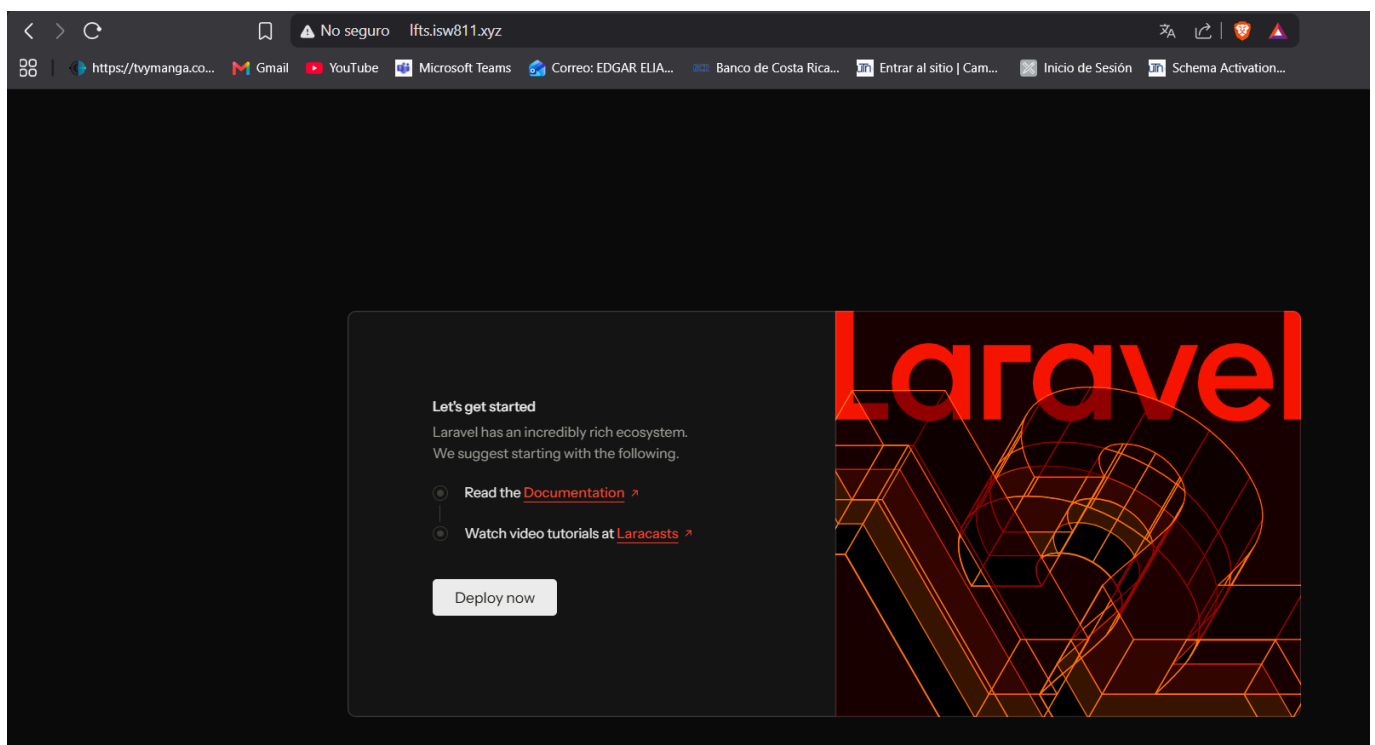
Significa:

- **Route** → Define una ruta dentro de la aplicación.
- **get()** → Indica el método HTTP que escuchará la ruta.
- **/** → Representa la URL que visitará el usuario.
- **function()** → Código que se ejecuta cuando se accede a la ruta.
- **view()** → Carga una vista que será mostrada al usuario.

```
routes > web.php
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4
5  Route::get('/', function () {
6      return view('welcome');
7  });
8
```

Vistas en Laravel

Las **vistas** son las páginas que el usuario observa en el navegador.

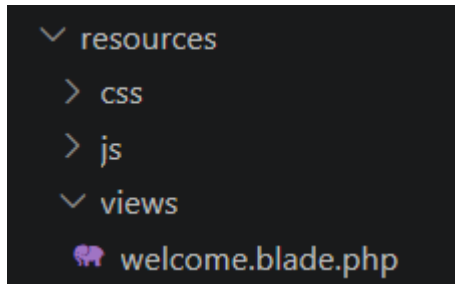


Normalmente se encuentran en:

```
resources/views
```

Ejemplo:

```
resources/views/welcome.blade.php
```



Laravel usa **Blade** como motor de plantillas para crear páginas HTML dinámicas.

Una vista puede contener:

- HTML
- CSS
- JavaScript
- Código Blade

Editar una vista

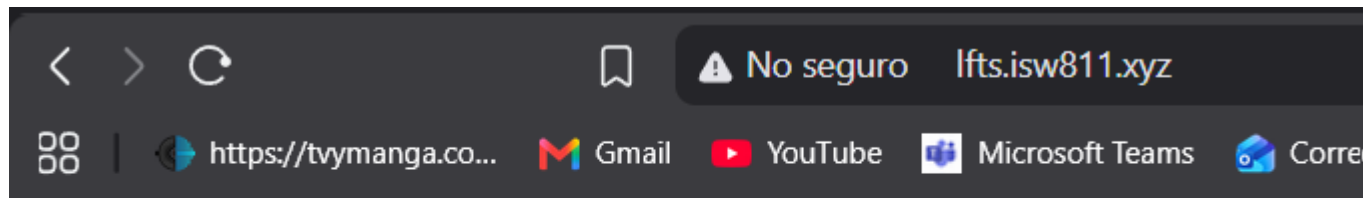
Las vistas se pueden modificar directamente.

Eliminamos todo el contenido y agregamos de ejemplo:

```
<h1>Hello World</h1>
```



Al guardar los cambios y actualizar el navegador, Laravel muestra automáticamente la nueva versión.



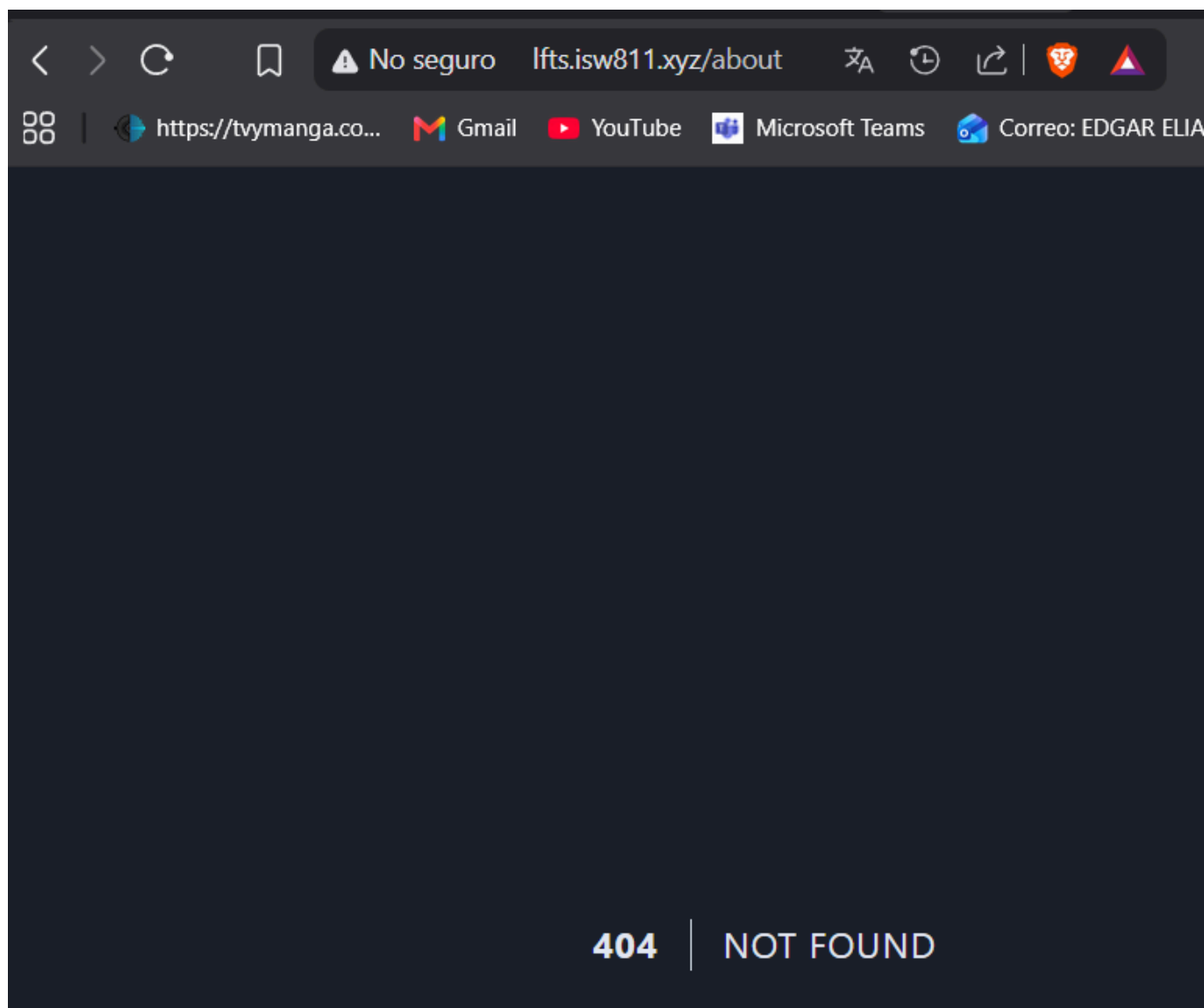
Welcome to Laravel

Crear enlaces entre páginas

Para navegar entre páginas se utiliza HTML:

```
<a href="/about">About Us</a>
```

Si la ruta no existe, Laravel muestra un error:



Esto ocurre porque la URL no está registrada.

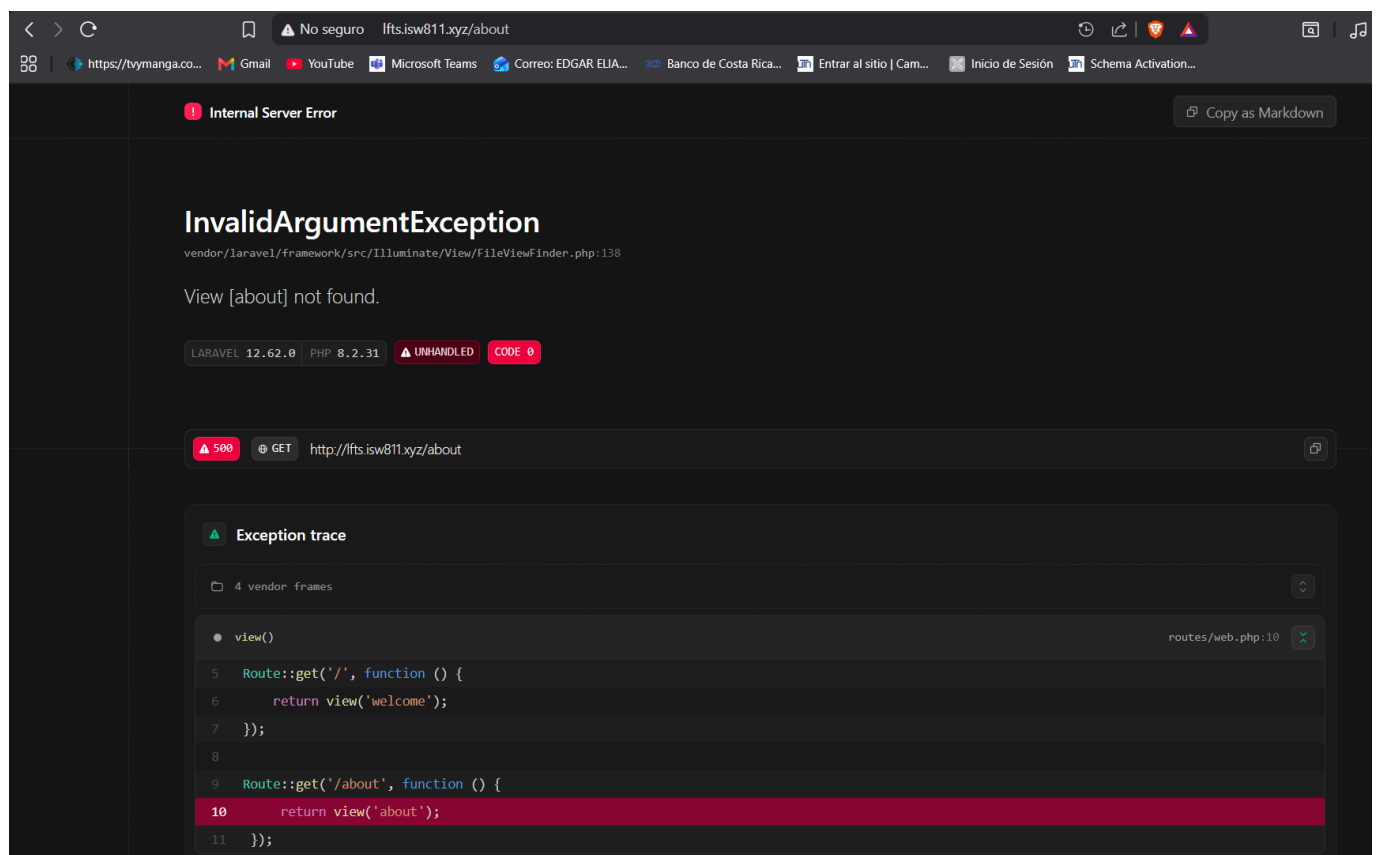
Crear una nueva ruta

Para crear una página **About**:

```
Route::get('/about', function () {  
    return view('about');  
});
```

Laravel buscará:

```
resources/views/about.blade.php
```



Retornar texto directamente

Una ruta también puede devolver texto:

```
Route::get('/about', function () {  
    return "About us";  
});
```

Aunque funciona, normalmente se recomienda utilizar vistas para páginas completas.

Crear una nueva vista

Si la vista no existe aparecerá:

```
View [about] not found
```

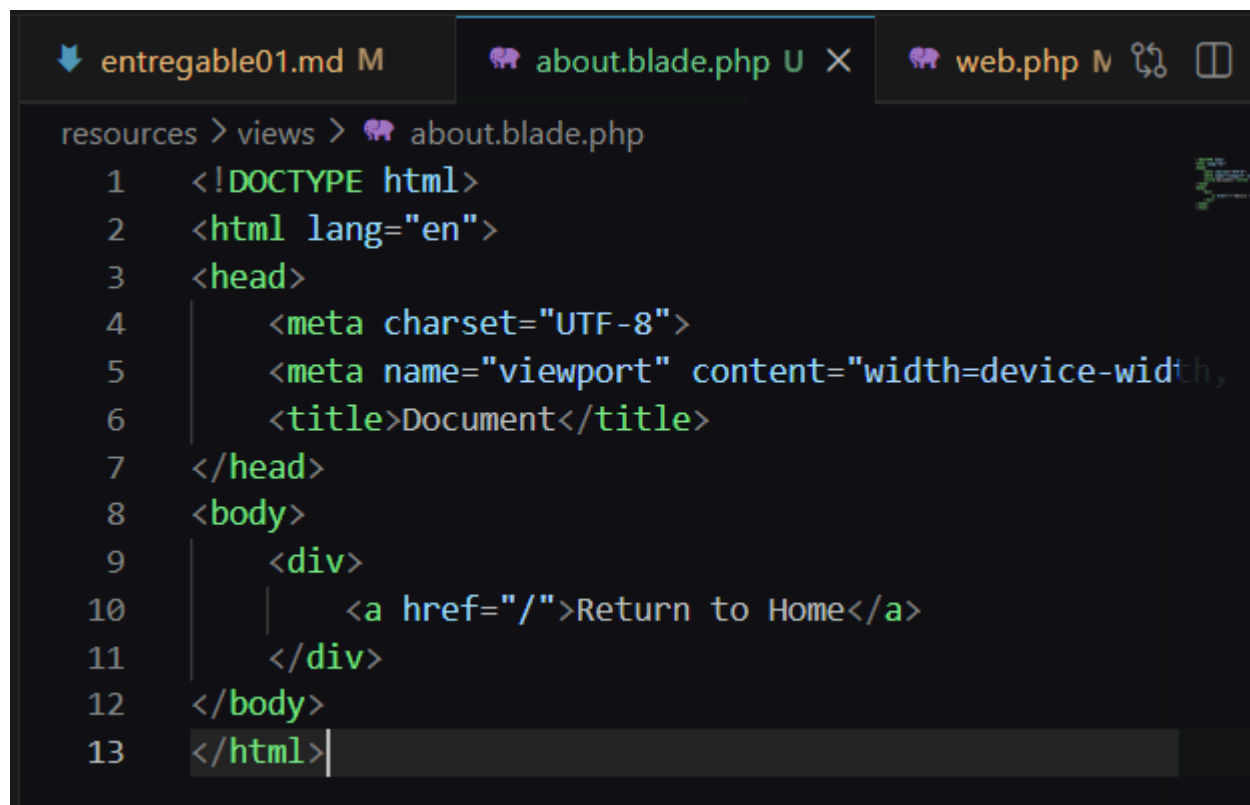
Se debe crear el archivo:

```
resources/views/about.blade.php
```

Ejemplo:

```
<h1>About Us</h1>

<a href="/">Return Home</a>
```



```
entregable01.md M  about.blade.php U X  web.php M  ↻  □
resources > views > about.blade.php
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width,
6      <title>Document</title>
7  </head>
8  <body>
9      <div>
10         <a href="/">Return to Home</a>
11     </div>
12 </body>
13 </html>
```

Ejercicio: Crear ruta Contact

Se debe crear una ruta que responda a:

```
GET /contact
```

Código:

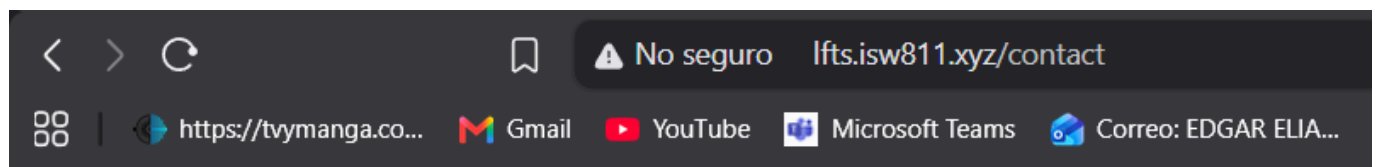
```
Route::get('/contact', function () {  
    return view('contact');  
});
```

Después crear:

```
resources/views/contact.blade.php
```

Ejemplo:

```
<h1>Contact</h1>  
  
<p>Email: example@test.com</p>
```



Contact Us

Email: example@test.com

[Return to Home](#)

Layout Files

Atajo `Route::view()`

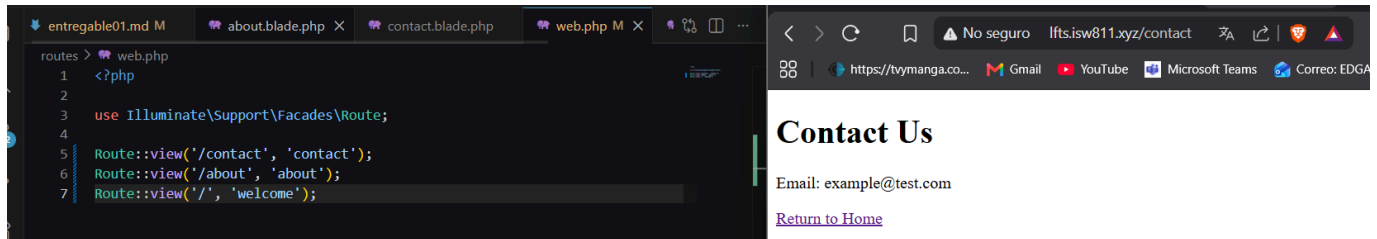
Cuando una ruta solo carga una vista, Laravel permite usar:

```
Route::view('/contact', 'contact');
```

En lugar de:

```
Route::get('/contact', function () {  
    return view('contact');  
});
```

Es útil para páginas estáticas.



Problema: Código repetido

Al crear varias páginas aparecen elementos repetidos:

- Menú
- HTML principal
- CSS
- JavaScript

Para evitarlo se utilizan layouts y componentes Blade.

Crear un Layout

Se crea un componente reutilizable: resources/views/components/layout.blade.php

Aquí se coloca el código común:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Document</title>  
</head>  
<body>  
    {{ $slot }}  
</body>  
</html>
```

Uso de Slot

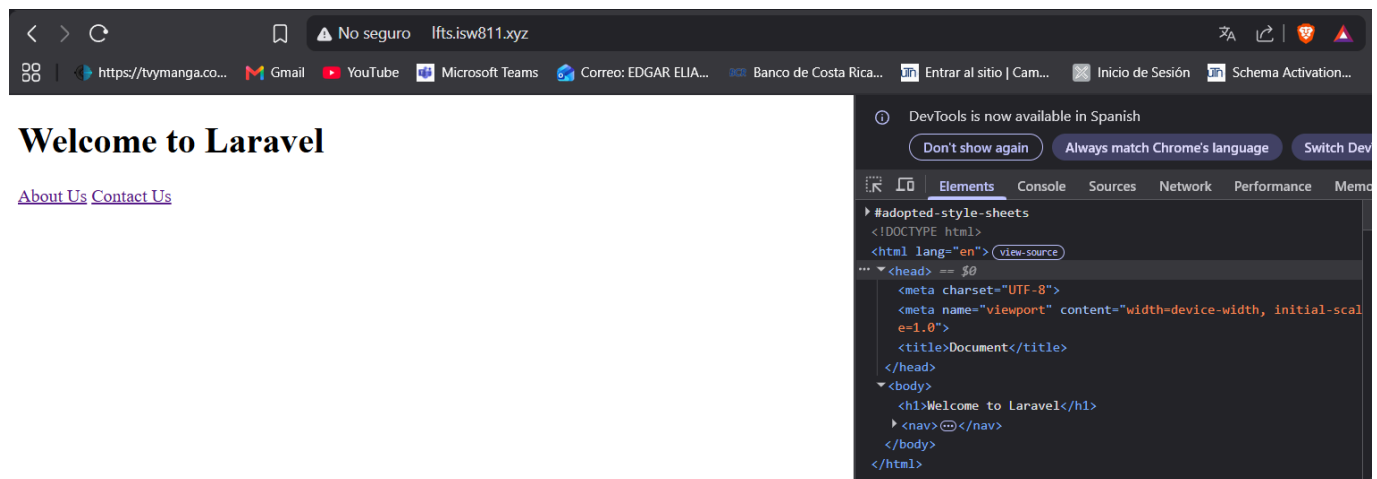
`{{ $slot }}` representa el contenido específico de cada página.

Ejemplo:


```
<x-layout>
  <h1>Welcome to Laravel</h1>

  <nav>
    <a href="about">About Us</a>
    <a href="contact">Contact Us</a>
  </nav>
</x-layout>
```

El contenido se inserta dentro del layout.



Props en componentes(Titulos)

Los componentes pueden recibir datos.

Queremos que nuestro titulos sean diferente segun la vista que ingresmos.

En el archivo layout.blade.php cambiamos el titulo por lo siguiente:

```
<title>{{ $title }}</title>
```

En cada vista se debe de colocar el titulo de la siguiente forma:

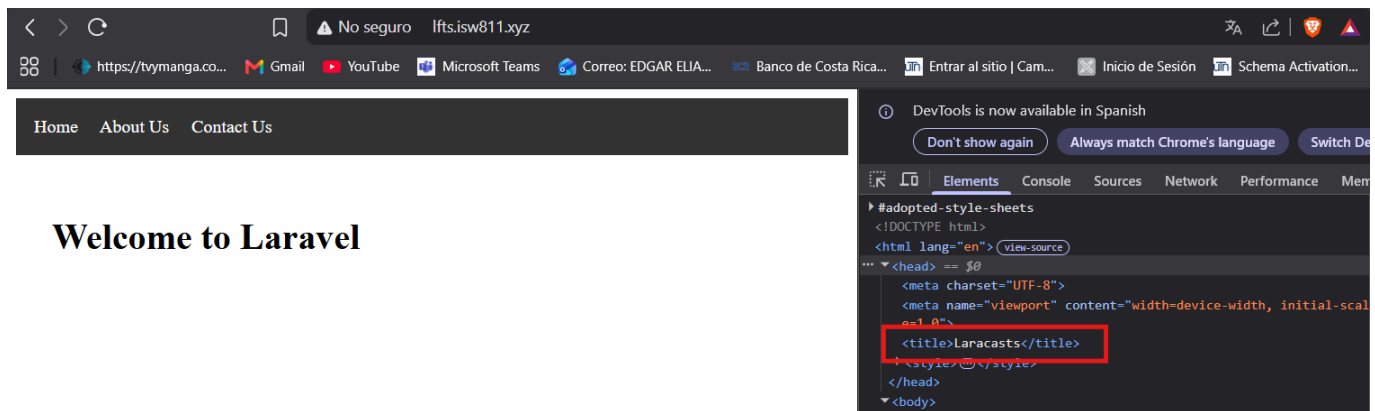
```
<x-layout title="About Us">
```

En el componente se define:

```
@props(['title'])
```

Esto permite crear valores dinámicos como títulos de página. También se pueden definir valores por defecto:

```
@props([
    'title' => 'Laracasts'
])
```



Attributes vs Props

Props — Son datos enviados al componente: title="Home"

Attributes — Son atributos HTML: class=""

id=""

Laravel los maneja de forma diferente.

Componentes reutilizables

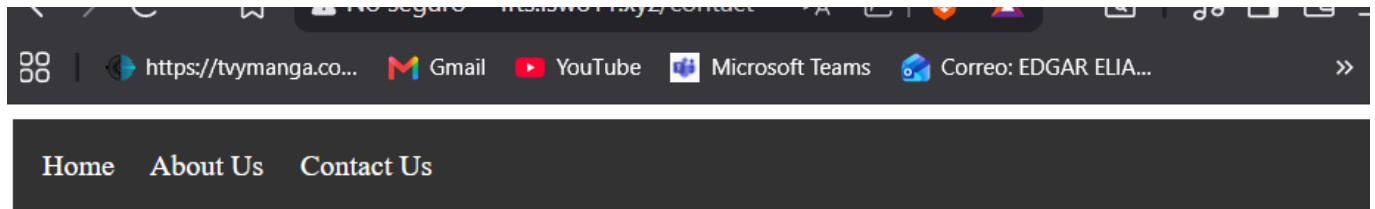
Se pueden crear componentes como tarjetas: components/card.blade.php

Ejemplo:

```
<div class="card" styles: >
  {{ $slot }}
</div>
```

Uso:

```
<x-card>
  Contenido
</x-card>
```



Contact Us

Contact us at: 11111111

Pass Data to Views

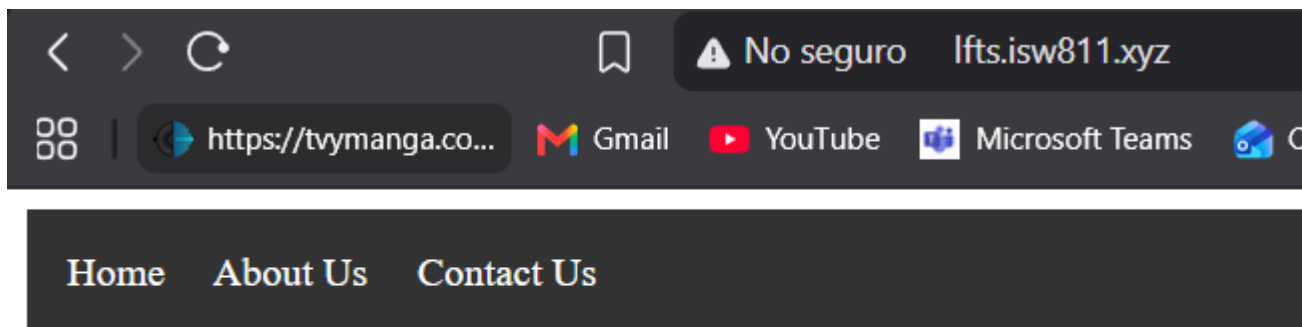
Métodos para pasar datos

En web.php **Con** `Route::view()` — tercer parámetro como arreglo:

```
Route::view('/welcome', 'welcome', ['greeting' => 'Hello, welcome to our website!', 'person' => 'Eliam!']);
```

En la vista de la pagina welcome:

```
<x-layout>
    {{ $greeting }}, {{ $person }}!
</x-layout>
```



Hello, welcome to our website!, Eliam!

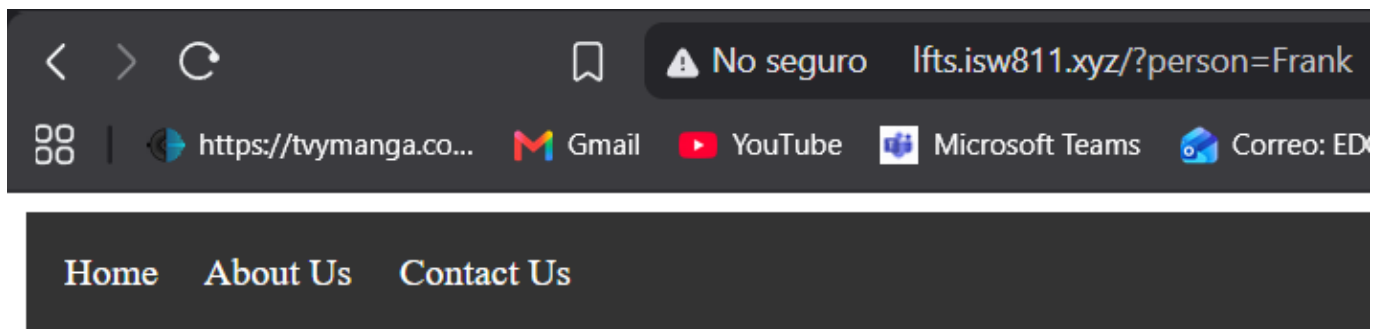
Con `Route::get()` — segundo argumento de `view()`:

```
Route::get('/', function () {  
    return view('welcome', ['person' => request('person')]);  
});
```

Query String

Leer parámetros de la URL con `request()`:

```
request('person') // lee ?person=Frank
```



Hello, welcome to our website!, Frank!

Valores por defecto

```
request('person', 'World') // si no hay valor, usa 'World'
```

Escapado y seguridad XSS

Sintaxis	Escapa HTML	Uso recomendado
{{ \$var }}	Sí	Datos del usuario
{!! \$var !!}	No	Solo datos de confianza

Otro metodo

```
Route::get('/', function () {  
    return view('welcome', [  
        'greeting' => 'Hello, welcome to our website!',  
        'person' => request('person')  
    ]);  
});
```

```
    });  
  });
```

Blade Directives

Pasar arreglos a las vistas

Se puede enviar un arreglo de datos a una vista así:

```
Route::get('/', function () {  
    return view('welcome', [  
        'tasks' => ['Go to the market', 'Walk the dog', 'Watch a video tutorial']  
    ]);  
});
```

No se puede hacer `{{ $tasks }}` directamente si es un arreglo — causará un error.

Internal Server Error

Copy as Markdown

TypeError

vendor/laravel/framework/src/Illuminate/Support/helpers.php:138

htmlspecialchars(): Argument #1 (\$string) must be of type string, array given

LARAVEL 12.62.0 PHP 8.2.31 UNHANDLED CODE 0

500 GET http://lfts.isw811.xyz

Exception trace

2 vendor frames

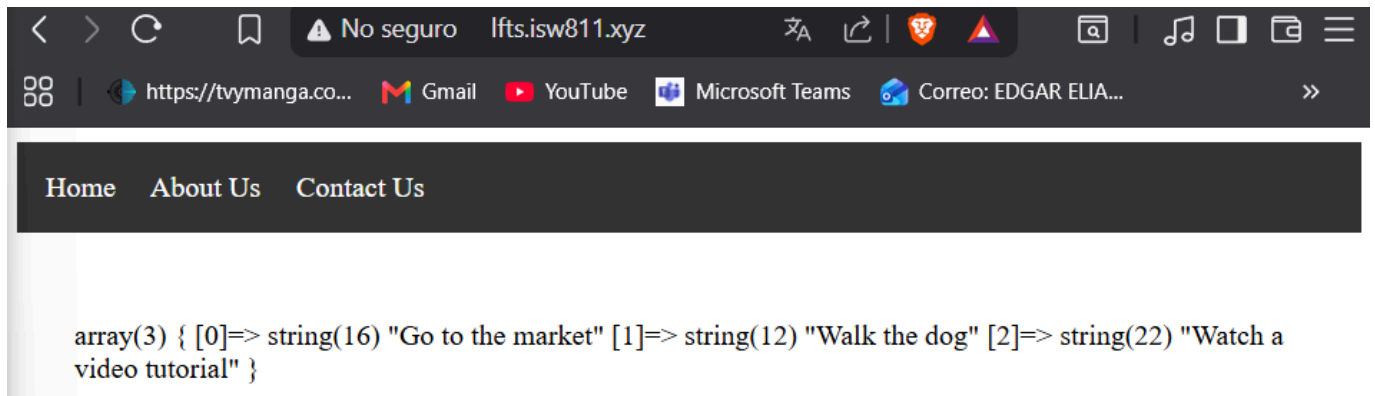
- e() resources/views/welcome.blade.php:3

```
1 <x-layout>
2 <p>
3 {{ $tasks }}
4 </p>
5 </x-layout>
```

55 vendor frames

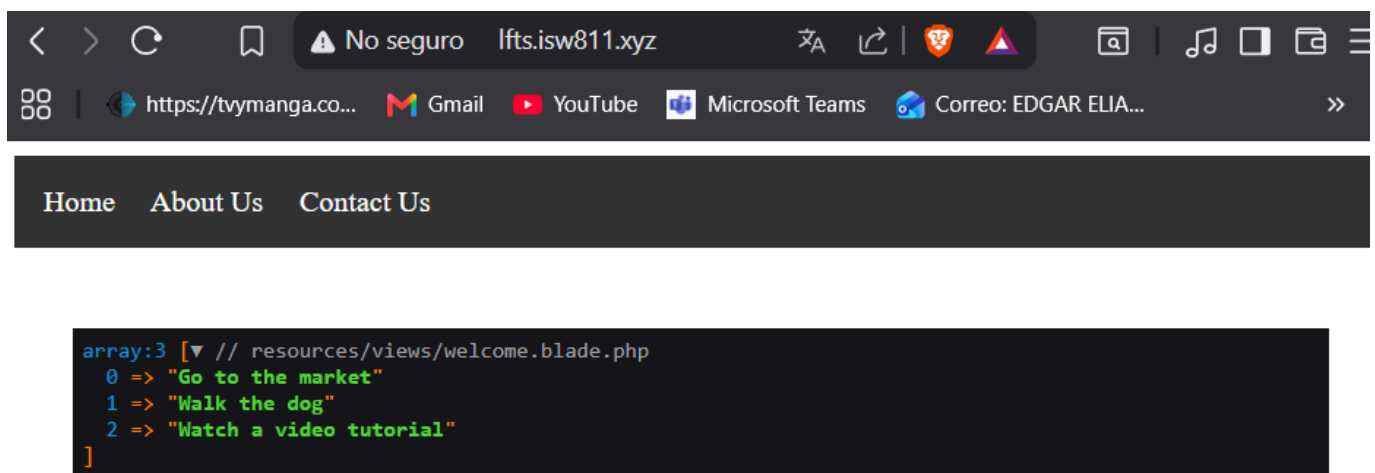
Depuración con `dump` y `dd`

Puedes utilizar `var_dump(tasks)` o `die(var_dump(tasks))`



En lugar de `var_dump()` o `die(var_dump())`, Blade ofrece directivas más limpias:

```
@dump($tasks)    {{-- muestra el contenido sin detener la ejecución --}}
@dd($tasks)      {{-- muestra el contenido y detiene la ejecución --}}
```



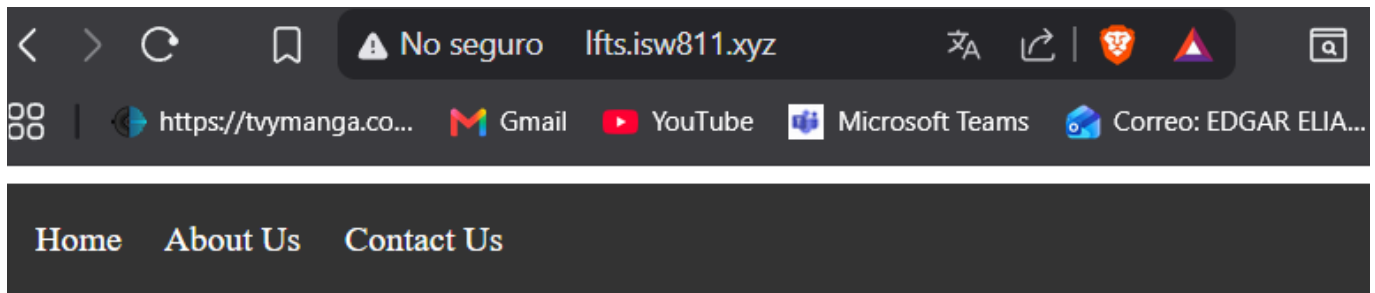
Welcome Eliam

Condicionales

```
<?php if (count($tasks)): ?>
    <p> Yes we have some tasks. How many Eliam? <?= count($tasks) ?> tasks, in
    fact.</p>
<?php endif; ?>
```

Si lo cambiamos por un formato blade debería de verse el mismo resultado:

```
@if (count($tasks))
    <p> Yes we have some tasks. How many Eliam? <?= count($tasks) ?> tasks, in
    fact.</p>
@endif
```



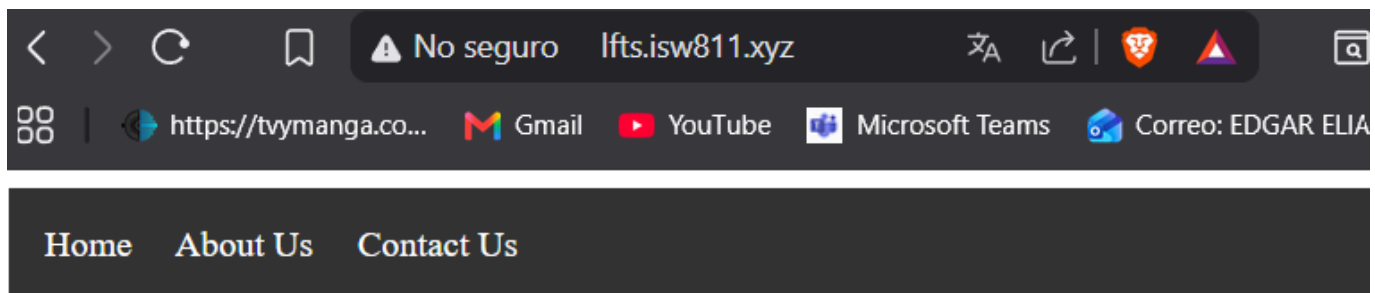
Yes we have some tasks. How many Eliam? 3 tasks, in fact.

También existe `@unless` como equivalente a `if not`:

```
@unless(count($tasks))
  No hay tareas activas.
@endunless
```

Bucles con `@foreach`

```
@foreach($tasks as $task)
  <li>{{ $task }}</li>
@endforeach
```



- Go to the market
- Walk the dog
- Watch a video tutorial

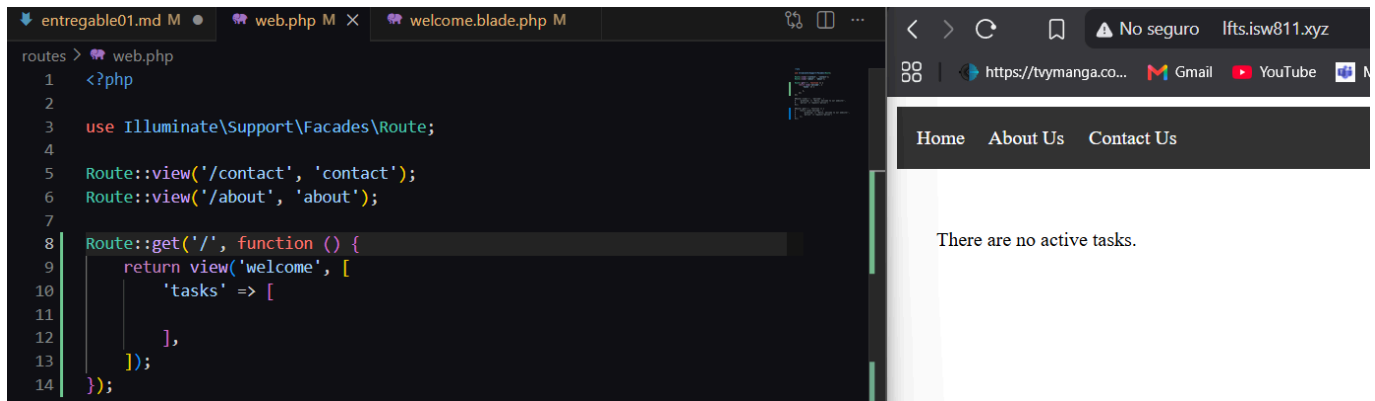
Manejo de arreglos vacíos con `@forelse`

Cuando el arreglo puede estar vacío, `@forelse` combina el bucle con un caso alternativo:

```
@forelse($tasks as $task)
  <li>{{ $task }}</li>
@empty
```



```
<p>There are no active tasks.</p>
@endforelse
```



Directivas de autenticación y autorización

Blade también incluye directivas para controlar acceso según el estado del usuario:

```
@auth
    {{-- usuario autenticado --}}
@endauth

@guest
    {{-- visitante sin sesión iniciada --}}
@endguest

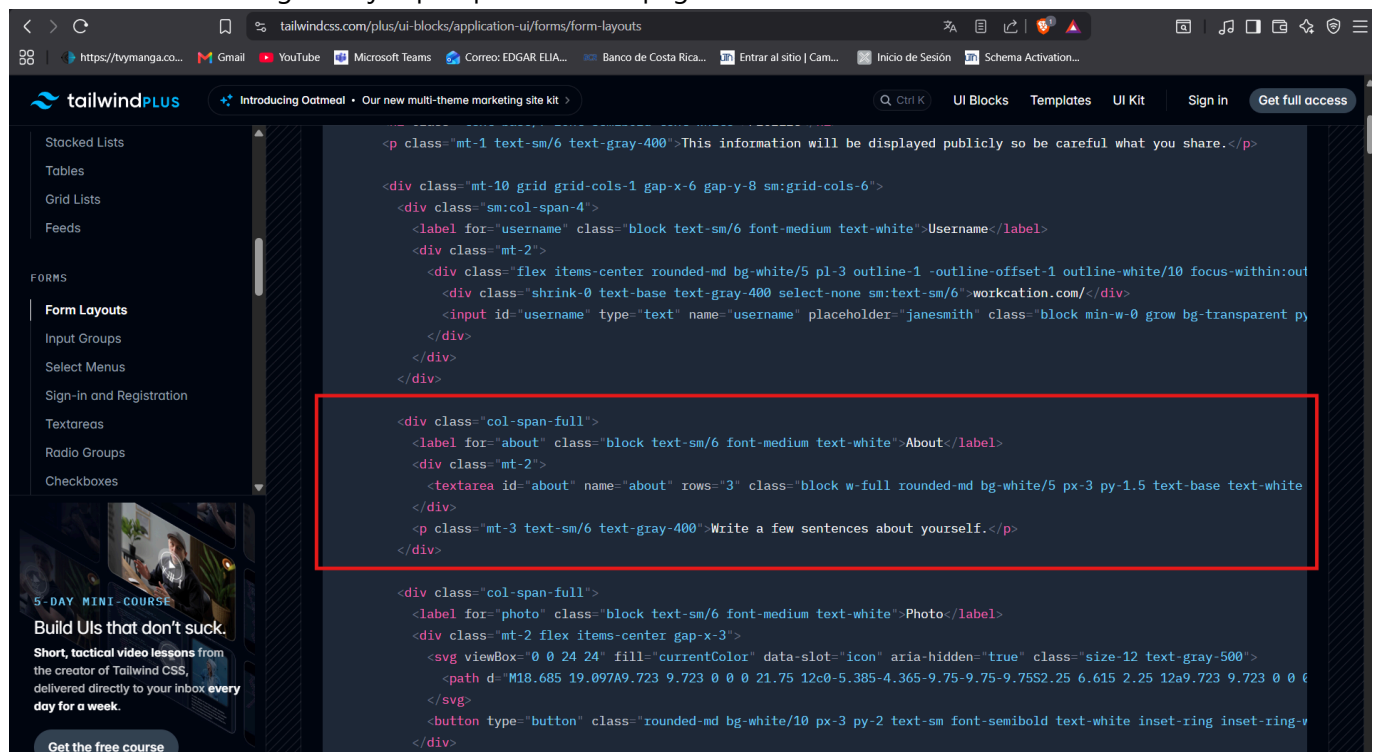
@can('editar', $post)
    <a href="/posts/edit">Editar</a>
@endcan
```

Forms

Crear la vista y el formulario

Se crea una vista `ideas.blade.php` con un formulario básico usando Tailwind CSS (<https://tailwindcss.com/>). El formulario debe tener `method="POST"` y un `action` que apunte al endpoint donde se enviará la data:

utilizaremos este codigo de ejemplo que esta en la pagina:



Home About Us Contact Us

About

Write a few sentences about yourself.

```

<form action="">
  <div class="col-span-full">
    <label for="idea" class="block text-sm/6 font-medium text-white">New
    idea</label>
    <div class="mt-2">
      <textarea id="idea" name="idea" rows="3" class="block w-full rounded-
      md bg-white/5 px-3 py-1.5 text-base text-white outline-1 -outline-offset-1
      outline-white/10 placeholder:text-gray-500 focus:outline-2 focus:-outline-offset-2
      focus:outline-indigo-500 sm:text-sm/6"></textarea>
    </div>
    <p class="mt-3 text-sm/6 text-gray-400">Have an idea to share?</p>
  </div>

  <div class="mt-6 flex items-center gap-x-6">

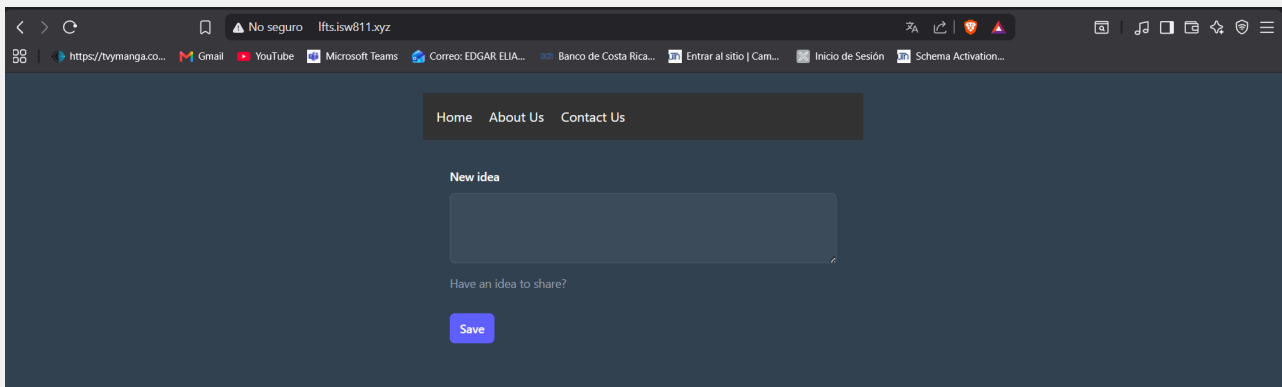
```

```
<button type="submit" class="rounded-md bg-indigo-500 px-3 py-2 text-sm font-semibold text-white focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-500">Save</button>
</div>
</form>
```

Agregamos esta línea en layout.blade.php:

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
```

Para estilos rápidos en demos se puede usar Tailwind desde un CDN, aunque en producción se recomienda instalarlo formalmente.



Registrar las rutas

Se necesitan **dos rutas** separadas: una para mostrar el formulario y otra para recibir los datos:

```
// Muestra el formulario
Route::get('/', function () {
    return view('ideas');
});

// Recibe y procesa el formulario
Route::post('/ideas', function () {
    dd('Hello');
});
```

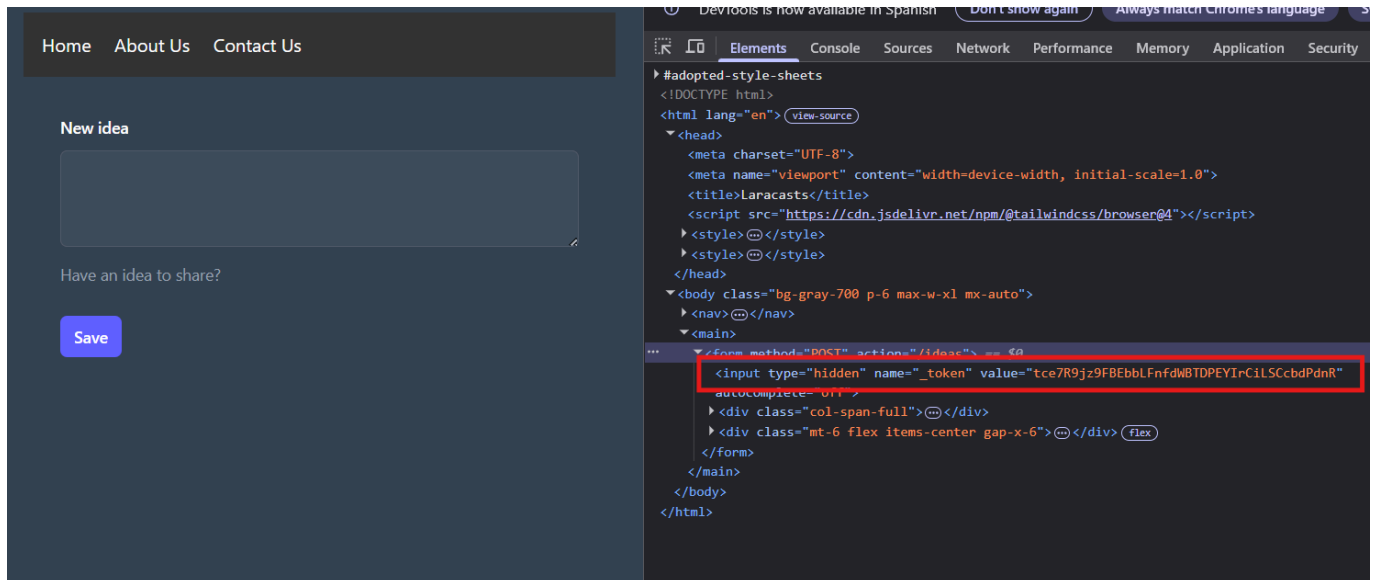
Protección CSRF

Al enviar un formulario POST, Laravel lanza un error **419** si no se incluye el token CSRF. La solución es agregar la directiva `@csrf` dentro del formulario:

```
<form method="POST" action="/ideas">
    @csrf
```

```
...
</form>
```

El token se genera en el servidor y se compara al recibir el formulario. Esto protege contra ataques de **Cross-Site Request Forgery**, donde un sitio externo podría enviar solicitudes maliciosas en nombre del usuario.



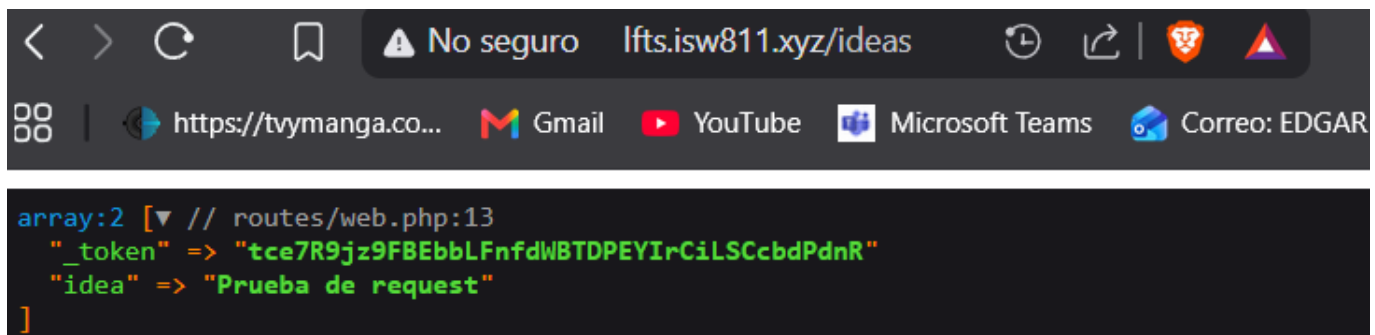
Leer datos del Request

Hay varias formas de acceder a los datos enviados:

```
// Función helper (la más simple)
request('idea')

// Facade de Laravel
Request::get('idea')

// Inyección de dependencia
function (Illuminate\Http\Request $request) {
    $request->get('idea');
}
```



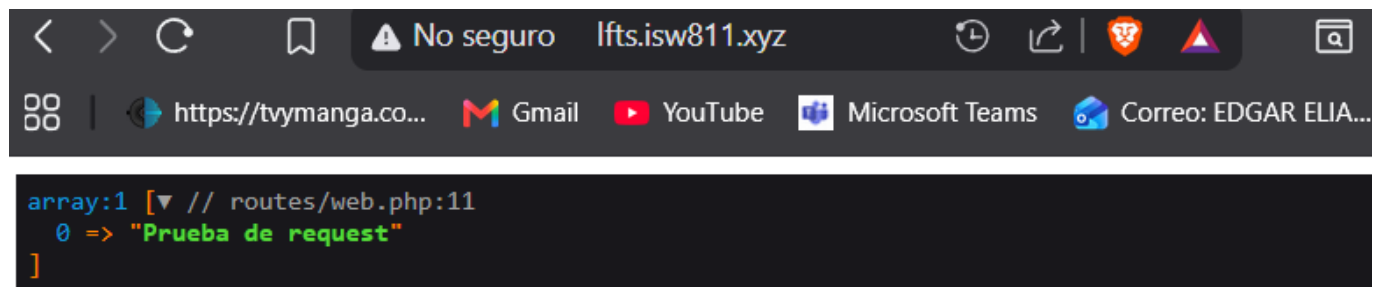
Guardar datos en Session

Como alternativa temporal a una base de datos, se puede usar la sesión del usuario:

```
// Agregar un item al arreglo 'ideas' en sesión
session()->push('ideas', $idea);

// Leer las ideas guardadas (con arreglo vacío como valor por defecto)
session('ideas', []);

// Eliminar las ideas de la sesión
session()->forget('ideas');
```

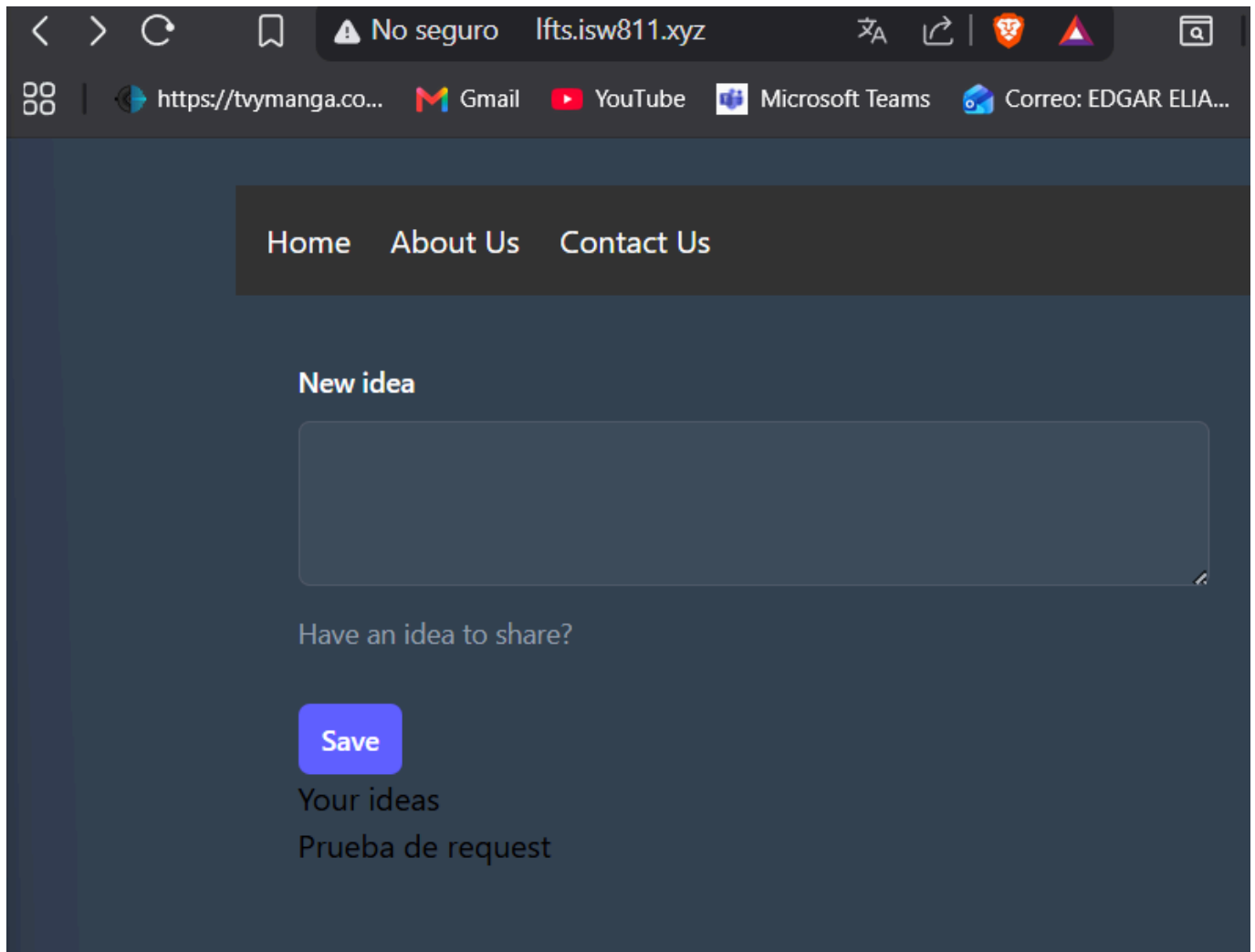


La sesión funciona como un bloc de notas único por usuario. Es temporal y no persiste entre navegadores distintos. Para persistencia real se debe usar una base de datos.

Mostrar las ideas guardadas

En la vista se recorre el arreglo con `@forelse` para manejar el caso vacío:

```
@if(count($ideas))
    <ul>
        @foreach($ideas as $idea)
            <li>{{ $idea }}</li>
        @endforeach
    </ul>
@endif
```



Databases, Migrations, and Eloquent

Configuración de la base de datos

Laravel soporta múltiples drivers de base de datos (MySQL, PostgreSQL, SQLite, entre otros). Por defecto usa **SQLite**, que es una base de datos basada en archivos. La configuración se define en el archivo `.env`:

```
DB_CONNECTION=sqlite
```

El archivo `.env` almacena la configuración del entorno local. En producción se usa un archivo `.env` distinto con configuraciones diferentes.

Migraciones

Las migraciones son **control de versiones para la base de datos**. Permiten definir tablas y columnas de forma programática y compartirlas con el equipo.

Crear una migración:

```
php artisan make:migration create_ideas_table
```

```
vagrant@webserver:~$ cd sites/lfts.isw811.xyz/  
vagrant@webserver:~/sites/lfts.isw811.xyz$ php artisan make:migration  
what should the migration be named?   
create_ideas_table
```

Estructura básica de una migración:

```
public function up(): void  
{  
    Schema::create('ideas', function (Blueprint $table) {  
        $table->id();  
        $table->text('description');  
        $table->timestamps();  
    });  
}  
  
public function down(): void  
{  
    Schema::dropIfExists('ideas');  
}
```

Ejecutar las migraciones:

```
php artisan migrate
```

Opciones para modificar una tabla existente

Opción 1 — Durante desarrollo, modificar la migración original y refrescar:

```
php artisan migrate:refresh # borra todos los datos y recrea las tablas
```

Opción 2 — En producción o trabajo en equipo, crear una nueva migración:

```
php artisan make:migration add_state_to_ideas_table
```

Consultas con el Facade DB

Forma genérica de consultar cualquier tabla:

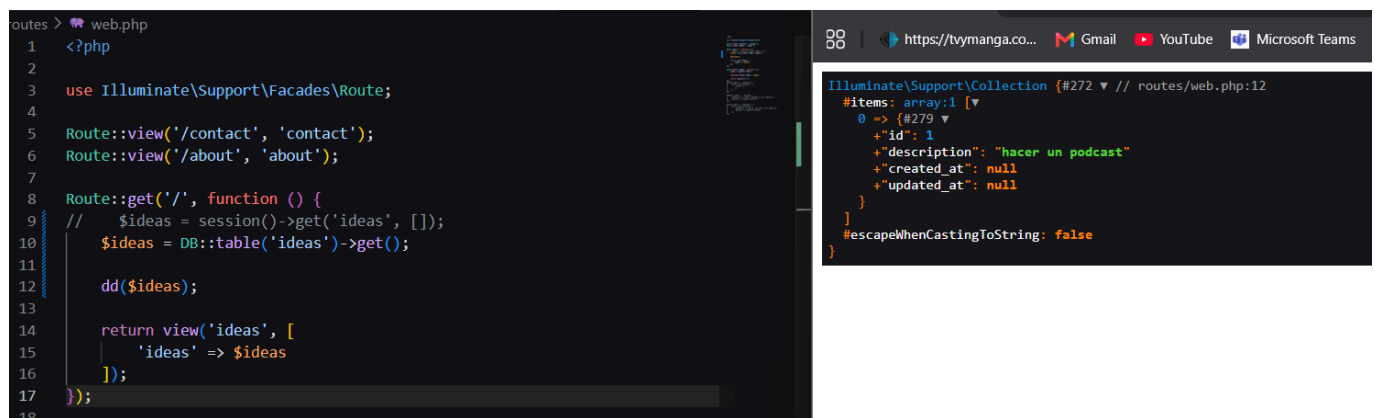
```
use Illuminate\Support\Facades\DB;

// Obtener todos los registros
$ideas = DB::table('ideas')->get();

// Con condición
$ideas = DB::table('ideas')->where('state', 'pending')->get();
```

Los resultados se retornan como una **colección de objetos**, no como arreglos:

```
$idea->description // acceder a una propiedad
```



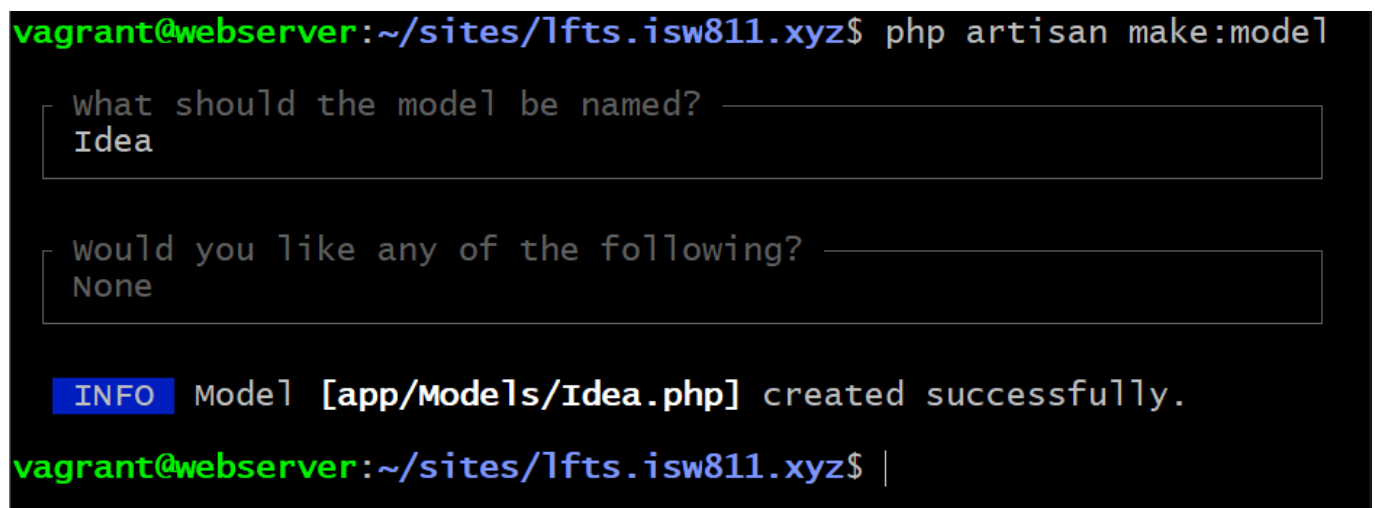
Eloquent ORM

Eloquent es el **ORM de Laravel**. Permite crear una clase que representa una tabla de la base de datos.

Crear un modelo:

```
php artisan make:model Idea
```

Esto genera `app/Models/Idea.php`:




```
class Idea extends Model
{
    protected $guarded = []; // deshabilitar protección para mayor control manual
}
```

Operaciones básicas con Eloquent

```
use App\Models\Idea;

// Obtener todos
$ideas = Idea::all();

// Buscar por ID
$Idea = Idea::find(1);

// Con condición
$ideas = Idea::where('state', 'pending')->get();

// Crear un nuevo registro
Idea::create([
    'description' => request('idea'),
    'state' => 'pending',
]);
```

Eloquent asigna automáticamente los campos `created_at` y `updated_at` al crear o actualizar registros.

Filtrado dinámico con Query String

Se puede filtrar resultados según parámetros de la URL (`?state=pending`):

```
$ideas = Idea::when(request('state'), function ($query, $state) {
    $query->where('state', $state);
})->get();
```

Ejemplos de uso en la URL:

```
/?state=pending    → solo ideas pendientes
/?state=completed  → solo ideas completadas
```

Flujo completo con Eloquent

```
// Mostrar ideas
Route::get('/', function () {
    $ideas = Idea::when(request('state'), function ($query, $state) {
        $query->where('state', $state);
    })->get();

    return view('ideas', ['ideas' => $ideas]);
});

// Guardar nueva idea
Route::post('/ideas', function () {
    Idea::create([
        'description' => request('idea'),
        'state' => 'pending',
    ]);

    return redirect('/');
});
```